Natural Language Processing (CS60075)

Assignment 1: N-gram Language Model

Total Marks : 50
Deadline: 14th Feb EOD <3

General Guidelines

- This assignment is to be done by the pre-decided student groups. You should not copy any code from other groups, or from any web source. We will use standard plagiarism detection tools on the submissions. Plagiarized codes will be awarded zero for the assignment, and we will not differentiate between who copied from whom.
- In this assignment, you are allowed to use Python libraries like spacy, NLTK, numpy and pandas for performing basic NLP tasks easily. However, **you are NOT allowed to use any feature / library / tool meant specifically for language modeling (or which allows you to build language models using built-in functions).**
- **Follow the submission instructions given at the end. Solutions should be uploaded via the CSE Moodle website.**
- You should organize your code into different functions and add explanatory comments. Make sure your code can be easily read. Codes that cannot be understood will be penalized.

  NOTE: Any queries about this assignment should be mailed to Rohit Dutta (rohitdutta2510@gmail.com).

Definition of an n-Gram Model

An **n-gram language model** is a probabilistic model used for predicting the next word in a sequence based on the previous (n−1) words. It relies on the **Markov assumption**, which states that the probability of a word depends only on a fixed number of preceding words, rather than the entire history of the sequence.

A language model assigns a probability to a sequence of words $W = (w_1, w_2, \ldots, w_T)$. Using the **chain rule of probability**, the probability of the full sequence can be decomposed as:

$$P(W) = P(w_1, w_2, \ldots, w_T) = \prod_{i=1}^{T} P(w_i \mid w_1, \ldots, w_{i-1})$$

However, computing $P(w_i|w_1, \ldots, w_{i-1})$ directly is impractical because it requires a vast amount of data to estimate probabilities for all possible word sequences. Instead, we approximate it using the **n-gram assumption**, which simplifies the dependency:

$$P(w_i \mid w_1, \ldots, w_{i-1}) \approx P(w_i \mid w_{i-(n-1)}, \ldots, w_{i-1})$$

where n is a fixed value determining how many preceding words are considered. This assumption leads to the **n-gram model**, where the probability of a sentence is computed as:

$$P(W) \approx \prod_{i=1}^{T} P(w_i \mid w_{i-(n-1)}, \ldots, w_{i-1})$$

Types of n-Gram Models

- **Unigram model ($n = 1$)**: Assumes each word is independent:
  $$P(W) = \prod_{i=1}^{T} P(w_i)$$
- **Bigram model ($n = 2$)**: Each word depends on the previous word:
  $$P(W) \approx \prod_{i=1}^{T} P(w_i|w_{i-1})$$
- **Trigram model ($n = 3$)**: Each word depends on the previous two words:
  $$P(W) \approx \prod_{i=1}^{T} P(w_i|w_{i-2}, w_{i-1})$$
- **Higher-order n-gram models ($n > 3$)**: Consider longer contexts but require more data for reliable estimation.

A. Dataset and Preprocessing **[10 Marks]**

This assignment uses a curated collection of text articles to train and evaluate language models under both in-domain and cross-domain settings. All data is provided in standard CSV format, where each row corresponds to a single article and contains two fields: *the article title and the text body.* Throughout the assignment, only the text body should be used for training and evaluation; the title field should be ignored.

The dataset is pre-split into *one training set* and *three independent test sets*, detailed as follows:

- **Training:** *'wiki_train.csv'* file contains ~14k articles.
- **Testing:** *'wiki_test.csv'*, *'arxiv_test.csv'* and *'financial_test.csv'* contains 100 articles from wikipedia, arxiv scientific abstracts and financial news.

Models should be ***trained only on the Wikipedia training set and evaluated separately on each test set.*** This evaluation protocol is designed to measure both performance on familiar data and the model's ability to generalize to unseen domains.

[TASKS]

Download the dataset from this link.

1. Extract 100 articles from the training set and keep them as the validation/development set.
2. Remove punctuations and non-ASCII characters (if any)
3. Stopword removal
4. Stemming/Lemmatization
5. Any other pre-processing steps you feel will improve efficiency and make the data more suitable for this type of language modeling.

B. Estimation Using Maximum Likelihood [25 Marks]

To compute $P(w_i|w_{i-(n-1)}, \ldots, w_{i-1})$, we use **Maximum Likelihood Estimation (MLE)**, which calculates probabilities based on relative frequencies:

$$P(w_i|w_{i-(n-1)}, ..., w_{i-1}) = \frac{C(w_{i-(n-1)}, ..., w_i)}{C(w_{i-(n-1)}, ..., w_{i-1})}$$

where:

- $C(w_{i-(n-1)}, ..., w_i)$ is the count of the sequence $(w_{i-(n-1)}, ..., w_i)$ in the training corpus.
- $C(w_{i-(n-1)}, ..., w_{i-1})$ is the count of the preceding $(n-1)$-gram.

Smoothing Techniques

One of the major challenges in n-gram modeling is **data sparsity**. Many valid word sequences may have zero probability because they are not observed in the training data. To address this, **smoothing techniques** are applied.. The simplest approach is **Laplace smoothing**, which adds a constant k (usually 1) to all counts:

$$P_{Lap}(w_i|w_{i-(n-1)}, ..., w_{i-1}) = \frac{C(w_{i-(n-1)}, ..., w_i) + k}{C(w_{i-(n-1)}, ..., w_{i-1}) + kV}$$

where:

- V is the vocabulary size.
- k is a small positive constant (usually $k = 1$).

Good Turing Discounting for Smoothing

Good--Turing smoothing is a technique for adjusting maximum likelihood estimates of n-gram probabilities to account for unseen events. Instead of relying on raw counts, it redistributes probability mass from frequently observed n-grams to rare and unseen n-grams.

Definition
Let:

- **c** be the observed count of an n-gram
- **Nc** be the number of distinct n-grams that occur exactly c times in the training corpus
- **N** be the total number of n-gram tokens in the corpus

Adjusted Count (Good–Turing Estimate)

For an n-gram with observed count **c**, the adjusted count **c*** is defined as:
c* = (c + 1) × (N(c+1) / Nc)

This replaces the original count c in probability estimation.

Special Case: Unseen n-grams (c = 0)

For n-grams that do not appear in the training corpus (c = 0):
c* = N1 / N0

Where:

- **N1** is the number of n-grams that occur exactly once
- **N0** is the total number of possible unseen n-grams

In practice, the total probability mass assigned to unseen n-grams is: **P(unseen) = N1 / N**

Probability Estimation

The Good–Turing probability of an n-gram is computed as:
P(w | h) = c*(h, w) / N(h)

Where:

- c*(h, w) is the Good–Turing adjusted count
- N(h) is the total number of times the history h occurs

For an example with calculations, visit Link to Video 1 / Link to Video 2

NOTE: Use P(unseen) = N1/N

[TASKS]

Train Unigram, Bigram and Trigram models using MLE

6. When creating the vocabulary of n-grams, select only those n-grams that appear in at least 1% articles of the train set
7. Apply Laplace smoothing to each of the models during MLE, take $k = 1$.
8. Apply Good-Turing smoothing to each of the models during MLE

C. Evaluating an n-Gram Model using Perplexity [10 Marks]

A key measure of language model performance is **perplexity**, which evaluates how well the model predicts a given test set. It is defined as:

$$PP(W) = \left(\prod_{i=1}^{T} P_{Lap}(w_i | w_{i-(n-1)}, ..., w_{i-1}) \right)^{-\frac{1}{T}}$$

Alternatively, using logarithms:

$$PP(W) = \exp\left(-\frac{1}{T} \sum_{i=1}^{T} \log P_{Lap}(w_i | w_{i-(n-1)}, ..., w_{i-1}) \right)$$

Lower **perplexity** indicates a better model.

[TASKS]

Evaluate the unigram, bigram, and trigram language models trained using Maximum Likelihood Estimation (MLE) with both **Laplace smoothing** and **Good–Turing smoothing** on the test set using **perplexity** as the evaluation metric. The evaluation should be conducted on **three test sets from different domains**: Wikipedia articles, financial news articles, and arXiv abstracts.

9. Compute perplexity of *each test set article* using the exp() formula for all the test sets.
10. Average across all articles in the test set to get overall perplexity for each model.
11. Based on the observed perplexity values across the three domains, analyze the impact of **domain shift** on language model performance (both **Laplace smoothing** and **Good–Turing smoothing**).
12. In particular, examine how perplexity varies when models trained on Wikipedia are evaluated on out-of-domain data, and identify which **n-gram order** exhibits the greatest degradation under domain shift.

Submission Instructions

Submit one .zip file containing the following. Name the compressed file the same as your roll number. Example: “25CS60R00.zip”'

1. **Codes:** Submit a main.ipynb notebook or a main.py file from where execution will be started. You are free to create other files as per convenience for legibility.
2. Do **NOT** include the **dataset folder**
3. **A report (as a doc or pdf)** containing: **[5 Marks]**
   - Short description of the work.
   - Report the time taken (in seconds) for each part of the code to run.

- Report the perplexity values for values for all the model combinations on all 3 test sets.
- Write all the observations from Section C in the report.
- Try to improve efficiency and/or performance of the system. For efficiency, both memory and time can be optimized. To this end, explain the different choices you've made and the reasons behind them. Marks will be given based on the creativity of the solution.